Mavi VPN Technical Whitepaper

Advanced Architecture, Protocol Engineering & Deep Packet Inspection Resistance

Mavi Development Team

July 17, 2026

Abstract

This whitepaper analyzes Mavi VPN, a Virtual Private Network architecture built primarily in Rust. It covers the IETF QUIC transport, the optional HTTP/2 CONNECT-IP transport over TLS/TCP, Encrypted Client Hello (ECH), MASQUE (RFC 9484), and the configurable MTU strategy. By using asynchronous I/O, dual-stack routing, and certificate pinning, Mavi VPN provides multiple transport choices for constrained network environments.

Contents

1 Introduction and Core Philosophy

Many VPN protocols expose recognizable wire patterns. WireGuard, for example, uses fixed-size handshake messages that a filtering system can fingerprint without decrypting their contents.

Mavi VPN discards these legacy assumptions. It operates on the principle that *all* traffic is subject to hostile interception, active probing, and Deep Packet Inspection (DPI). Its QUIC censorship-resistant mode uses HTTP/3 framing and probe camouflage; its separate HTTP/2 mode provides a reliable TLS/TCP CONNECT-IP fallback without claiming to be ordinary browser traffic.

1.1 Key Design Goals

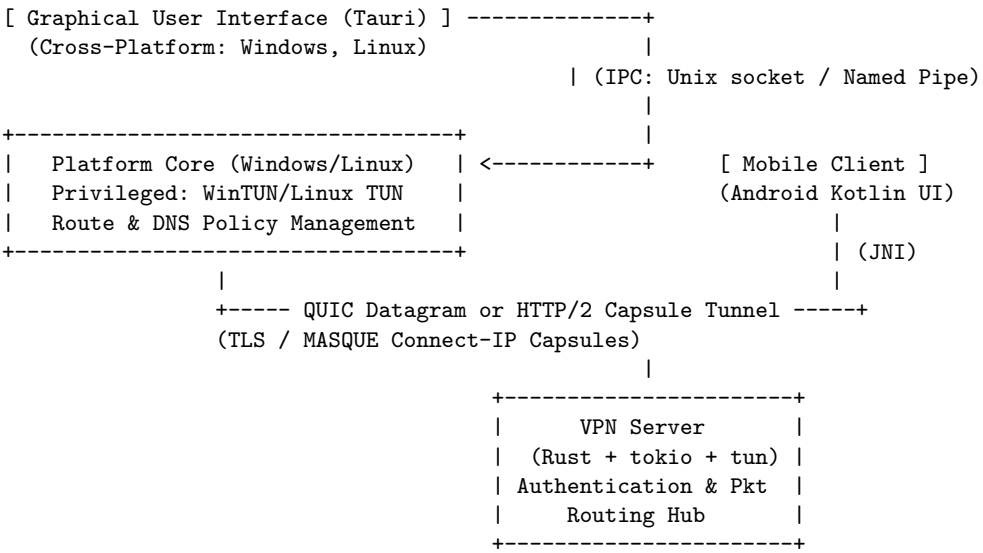
- **Protocol Camouflage**: Using HTTP/3 application-layer framing (ALPN `h3`), a cover SNI, and desktop ECH GREASE based on RFC 9849. ECH uses HPKE as specified by RFC 9180.
- **Performance**: Using Rust's `tokio` asynchronous runtime, `bytes`-backed packet buffers, and hardware-accelerated cryptography through `aws-lc-rs`.
- **Network Resilience**: Supporting QUIC connection migration for mobile roaming and BBR congestion control for high-bandwidth or high-latency paths.
- **Memory Safety**: Rust prevents many buffer-overflow and use-after-free defects in safe code. Platform APIs, dependencies, configuration, and explicitly audited unsafe boundaries still require security review.

2 System Architecture and Topology

The ecosystem is separated into a backend, platform-specific privileged network services, and unprivileged user interfaces.

2.1 Architectural Component Layout

The architecture separates unprivileged UI tasks from highly privileged network stack manipulations using local Inter-Process Communication (IPC).



2.2 The Backend (Server-Side)

The Linux-based backend is the routing hub. It terminates QUIC and optional HTTP/2 connections, verifies identity tokens or OpenID Connect (OIDC) JWTs, allocates private Dual-Stack IP addresses, and routes packets between the inner virtual network and the internet via Network Address Translation (NAT).

The server uses Rust's platform-default allocator; it does not configure a custom global allocator. Packet handling uses `bytes`-backed buffers and bounded asynchronous queues. Throughput and allocation behavior must be established with workload-specific benchmarks rather than allocator guarantees.

2.3 Platform-Specific Clients

Mavi VPN provides native drivers rather than relying on generic user-space wrappers:

- **Windows (`windows` crate)**: Uses the **WinTUN** ring-buffer adapter. Route management is enacted via the Win32 API, and DNS policy is applied through the Windows **Name Resolution Policy Table (NRPT)** so matching queries use the tunnel resolver.
- **Linux (`linux` crate)**: Directly interacts with `/dev/net/tun` via low-level `ioctl` calls using the `tun` abstraction. It employs `systemd-resolved` interfacing to push the VPN DNS resolver to the top of the OS lookup table.
- **Android (`android` crate)**: Passes the Android `VpnService` file descriptor through JNI into the Rust asynchronous event loop via `tokio::io::unix::AsyncFd`. After setup, per-packet polling remains in the native Rust path.

3 Transport & Framing: The QUIC Paradigm

Carrying a Layer 3 VPN over TCP can create nested retransmission and head-of-line blocking when inner TCP traffic experiences loss. Raw UDP, by contrast, leaves congestion control and encryption to the application protocol.

Mavi VPN leverages **QUIC** by default. We use the `quinn` Rust crate (implementation of IETF QUIC) to provide independent, unreliable datagrams in an encrypted, congestion-controlled wrapper. An optional HTTP/2 CONNECT-IP mode uses TLS/TCP and RFC 8441 Extended CONNECT with RFC 9297 capsules; its capsule delivery is reliable and ordered.

3.1 Datagrams vs. Streams

- **The Data Plane (Datagrams)**: Encapsulated IP packets (IPv4/IPv6) are transmitted using QUIC Datagrams (RFC 9221). Datagrams are unordered and unreliable, so loss of one VPN packet does not block later packets on a QUIC stream; the inner protocol remains responsible for recovery.
- **The Control Plane (Streams)**: Configuration and authentication messages are sent over reliable, ordered QUIC streams before the client configures its virtual interface.

3.2 HTTP/2 CONNECT-IP Fallback

When `VPN_HTTP2_BIND_ADDR` is configured, the backend exposes a separate TLS/TCP listener that negotiates ALPN `h2`. Clients send an Extended CONNECT request to `/.well-known/masque/ip/*/*/` with protocol `connect-ip` and `capsule-protocol: ?1`. Address, route, configuration, reauthentication, and IP packet data are exchanged as CONNECT-IP capsules inside HTTP/2 DATA frames. HTTP/2 mode is mutually exclusive with QUIC HTTP/3 framing, censorship-resistant mode, and ECH.

3.3 Congestion Control: BBR (Bottleneck Bandwidth and RTT)

Mavi VPN configures **BBR** congestion control for QUIC. BBR estimates bottleneck bandwidth and baseline RTT rather than reacting only to individual losses. Its effectiveness depends on the network path and must be measured under the intended workload.

4 Censorship Resistance & DPI Evasion

Filtering systems can combine entropy analysis, handshake profiling, traffic shape, and active probes. Mavi VPN provides several countermeasures, but no camouflage mode can guarantee indistinguishability against every observer.

4.1 Encrypted Client Hello (ECH — RFC 9849; HPKE — RFC 9180)

In standard TLS 1.3, the Server Name Indication (SNI) is transmitted in plaintext, allowing firewalls to block connections to known VPN endpoints instantly.

- Windows and Linux clients parse an administrator-provided hex `ECHConfigList`, use its `public_name` as a cover SNI, and configure rustls `EchMode::Grease` through the `aws-lc-rs` HPKE implementation.
- Android parses the same `public_name` and uses it as its SNI, but its `ring` crypto provider lacks HPKE and therefore does not emit an ECH extension.
- The backend generates and persists ECH config and key artifacts, but rustls server-side ECH acceptance is not wired in this release. It does not decrypt an inner ClientHello.
- Consequently, the current feature provides ClientHello camouflage and ECH compatibility testing, not full end-to-end ECH confidentiality.

4.2 Active Probe Emulation (The Fake Nginx Server)

If a DPI system (or a malicious actor) strips the ECH payload, guesses the endpoint, or initiates a raw scan on the UDP port, a standard VPN would drop the packet or issue a QUIC `CONNECTION_CLOSE`, which immediately identifies it as a non-standard service.

Mavi VPN performs **Active Probe Emulation**. When an incoming connection fails authentication or presents an invalid ALPN, the backend does not silently drop it. Instead:

1. A grace period of 50ms (`H3_DETECTION_GRACE`) is allocated to allow stream multiplexing.
2. The engine shifts the connection context natively into an `h3` (HTTP/3) handler.
3. It creates a control stream (`0x00 0x04 0x00`) representing HTTP/3 settings.
4. It sends raw, QPACK-encoded headers equivalent to `:status 200` and `server: nginx`.
5. It flushes an HTML layout representing the default "Welcome to nginx!" landing page directly onto the bidirectional QUIC stream.

This behavior can reduce the value of simple active probes. Statistical or adaptive classifiers may still distinguish VPN traffic from ordinary browsing.

4.3 MASQUE (RFC 9484) Connect-IP Encapsulation

The "HTTP/3 Framing" flag uses an IETF-aligned MASQUE CONNECT-IP request and capsule data model.

1. The client establishes a genuine HTTP/3 Request over a bidirectional QUIC stream: `CONNECT /.well-known/masque/ip/*/*/ HTTP/3` with `:protocol=connect-ip`.
2. Tunnel configuration uses HTTP/3 **Capsules**. Mavi uses `ADDRESS_ASSIGN` (0x01) and `ROUTE_ADVERTISEMENT` (0x03) to allocate IPs and advertise routes.
3. A proprietary vendor capsule, `MAVI_CONFIG` (0x4D56), transmits supplementary configuration (DNS parameters, Pinned MTU metadata).
4. Data datagrams are prefixed with the RFC 9484 Quarter Stream ID and Context ID (`0x00 0x00`).

5 Authentication and Cryptography

Pinned deployments do not rely solely on the public Web PKI trust store. A client accepts the configured leaf-certificate digest, so secure pin distribution and rotation are part of the deployment's trust model.

5.1 SHA-256 Certificate Pinning

- During backend deployment, the server generates a TLS certificate and its private key.
- It immediately computes the SHA-256 fingerprint hash of the DER-encoded certificate and stores it in `cert_pin.txt`.
- The client compares the leaf certificate's SHA-256 digest with one or more configured pins in constant time. A mismatch aborts the handshake with a certificate-pin error. This protects against an untrusted or compromised CA only when pin distribution and the client configuration remain trustworthy; it does not protect a compromised endpoint or stolen server key.

5.2 Identity Provider Integration (Keycloak OIDC)

While static tokens are suitable for private deployments, Mavi VPN's architectural scale targets enterprise environments requiring complex Identity and Access Management (IAM).

- The platform integrates native **Proof Key for Code Exchange (PKCE) OAuth2** flows.
- The desktop client initializes a temporary loopback callback listener on `localhost:18923` and opens the machine's default browser at the Keycloak authorization endpoint.
- Upon successful SSO login, the IdP redirects back to the loopback listener with an authorization code.
- The VPN backend securely validates the resulting JSON Web Token against Keycloak's cryptographic JWKS (JSON Web Key Set) endpoint, enforcing `azp` (Authorized Party) checks in constant time.

6 The "Pinned MTU" Engineering Strategy

MTU mismatch is a frequent source of VPN black holes. Exceeding the available path MTU can lead to dropped packets when discovery or fragmentation does not succeed.

6.1 Evaluating Path MTU Discovery (PMTUD)

QUIC can probe larger datagram sizes to discover the available path. Access technologies such as PPPoE or additional tunnels can reduce the usable MTU below Ethernet's typical 1500 bytes. Fragmented UDP is not reliable across every network or filtering device, which can turn an oversized encapsulated packet into a silent failure.

6.2 Dual-Boundary Pinned Limitation

Mavi VPN bounds PMTUD variance with an operator-selected inner MTU in the range 1280–1360. For QUIC transports the outer payload MTU is derived as `VPN_MTU + 80`; HTTP/2/TCP has no QUIC payload MTU.

1. The Outer QUIC Payload: Derived from the Inner MTU

We actively disable PMTUD within the Quinn stack:

```
transport_config.mtu_discovery_config(None);
transport_config.initial_mtu(vpn_mtu + 80);
transport_config.min_mtu(vpn_mtu + 80);
```

With the default `VPN_MTU=1280`, the QUIC payload limit is 1360 bytes. The derived value keeps the encrypted UDP packet below common 1460-byte paths.

2. The Inner Payload Tunnel: Configurable at 1280–1360 Bytes

1280 Bytes is the minimum IPv6 link MTU. Configuring the virtual TUN adapter within the supported range tells the Windows, Linux, or Android network stack which maximum packet size the tunnel accepts.

3. The Protocol Mathematics

- Default QUIC Payload: $1280 + 80 = 1360$ Bytes.
- Transport Overhead: ∼80 Bytes (40B for IPv6 + 8B for UDP + 32B for QUIC Headers/TLS Auth Tag).
- Available Payload: `VPN_MTU` Bytes.
- Max Inner Packet: the selected TUN MTU.

This sizing reduces fragmentation risk on common access networks, but it cannot guarantee a fragmentation-free path on every network.

6.3 ICMP Asynchronous Emulation

If the local stack still produces a packet larger than the configured TUN MTU, the Mavi VPN core intercepts it rather than forwarding it. The core extracts the source/destination IPs and dynamically synthesizes an **ICMPv4/ICMPv6 "Packet Too Big" (PTB)** error payload directly inside the event loop, immediately injecting it back into the local operating system's ingress queue. This forces the host OS to accurately clamp its TCP Maximum Segment Size (MSS) without requiring messy server-side `iptables mangle` hacks.

7 System Optimization and Kernel Offloading

The implementation configures socket buffers and transport offload features to reduce overhead on high-throughput links.

7.1 Socket Buffers and GSO

Larger UDP socket buffers can absorb short bursts when the application or network stack cannot drain the queue immediately.

- Mavi VPN requests 4 MB `SO_SNDBUF` and `SO_RCVBUF` socket buffers; the effective size remains operating-system dependent.
- **Generic Segmentation Offload (GSO)** is enabled in the transport configuration. When the operating system and network device support it, GSO can combine writes and reduce per-packet syscall overhead.

7.2 IpGuard RAII and Dual-Stack Operations

Server-side IP lease tracking uses RAII (Resource Acquisition Is Initialization). After authentication, an `IpGuard` owns the assigned IPv4 and IPv6 leases. When a session ends—through a clean disconnect, error, or timeout—dropping the guard returns those leases to the pool. The network natively supports **Dual-Stack** deployments, assigning a dedicated `fd00::/64` IP and routing it accurately using `ip6tables` NAT66 masquerading to the WAN block.

8 Conclusion

The Mavi VPN architecture combines raw QUIC, HTTP/3/MASQUE camouflage, and an optional HTTP/2 CONNECT-IP fallback. Through careful packet validation, certificate pinning, authentication, and transport-specific MTU handling, Mavi VPN delivers high-performance edge routing while making the deployment trade-offs explicit: QUIC provides unreliable datagrams and roaming, while HTTP/2 provides reliable ordered capsules over TLS/TCP.